DESPLIEGUE DE APLICACIONES WEB — CURSO 2026/2027

Servidor de aplicaciones Ejecución, estado y rendimiento

Tema 3 — Publicación y ejecución de aplicaciones web



Objetivo de la sesión

Punto de partida

Ya publicamos y observamos la aplicación.

Ahora miramos dentro del proceso que atiende las peticiones.

Preguntas de la sesión

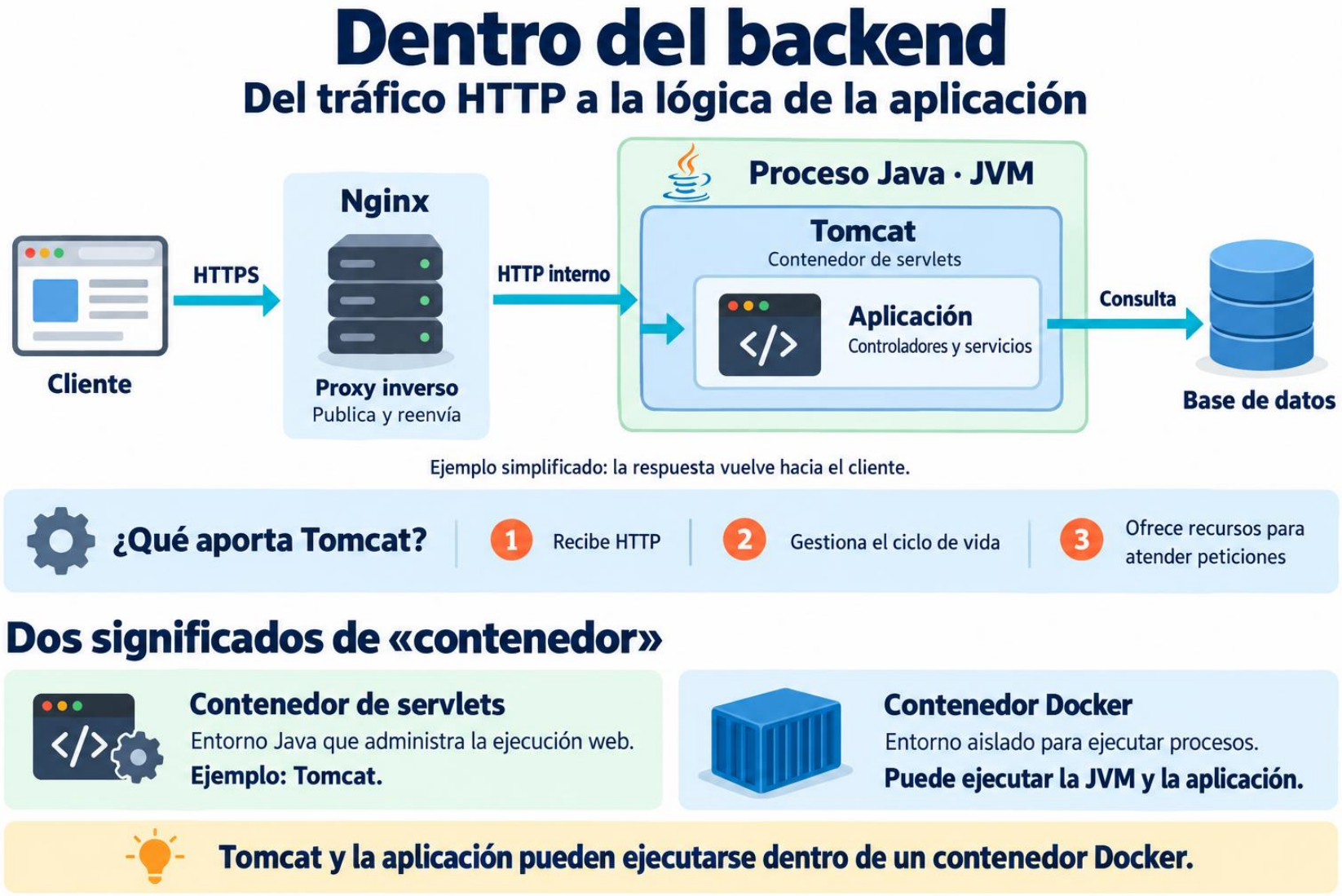
¿Qué administra el servidor?

¿Dónde vive la sesión del usuario?

¿Qué limita la mejora al añadir réplicas?

La ejecución, el estado y los recursos condicionan el comportamiento del backend.

Dentro del backend



Contenedor de servlets y concurrencia

Administración de la aplicación

Crear el contexto web.

Inicializar y retirar componentes.

Atender HTTP y gestionar sesiones.

Recursos de ejecución

El servidor reutiliza recursos, como hilos.

Las peticiones comparten CPU y memoria.

Los límites pueden provocar colas, rechazos o timeouts.

Una petición puede estar ejecutándose o esperando un recurso.

WAR, imagen y contenedor

Artefacto WAR

Empaqueta la aplicación web Java y sus recursos.

Su extensión no garantiza que sea ejecutable con java -jar.

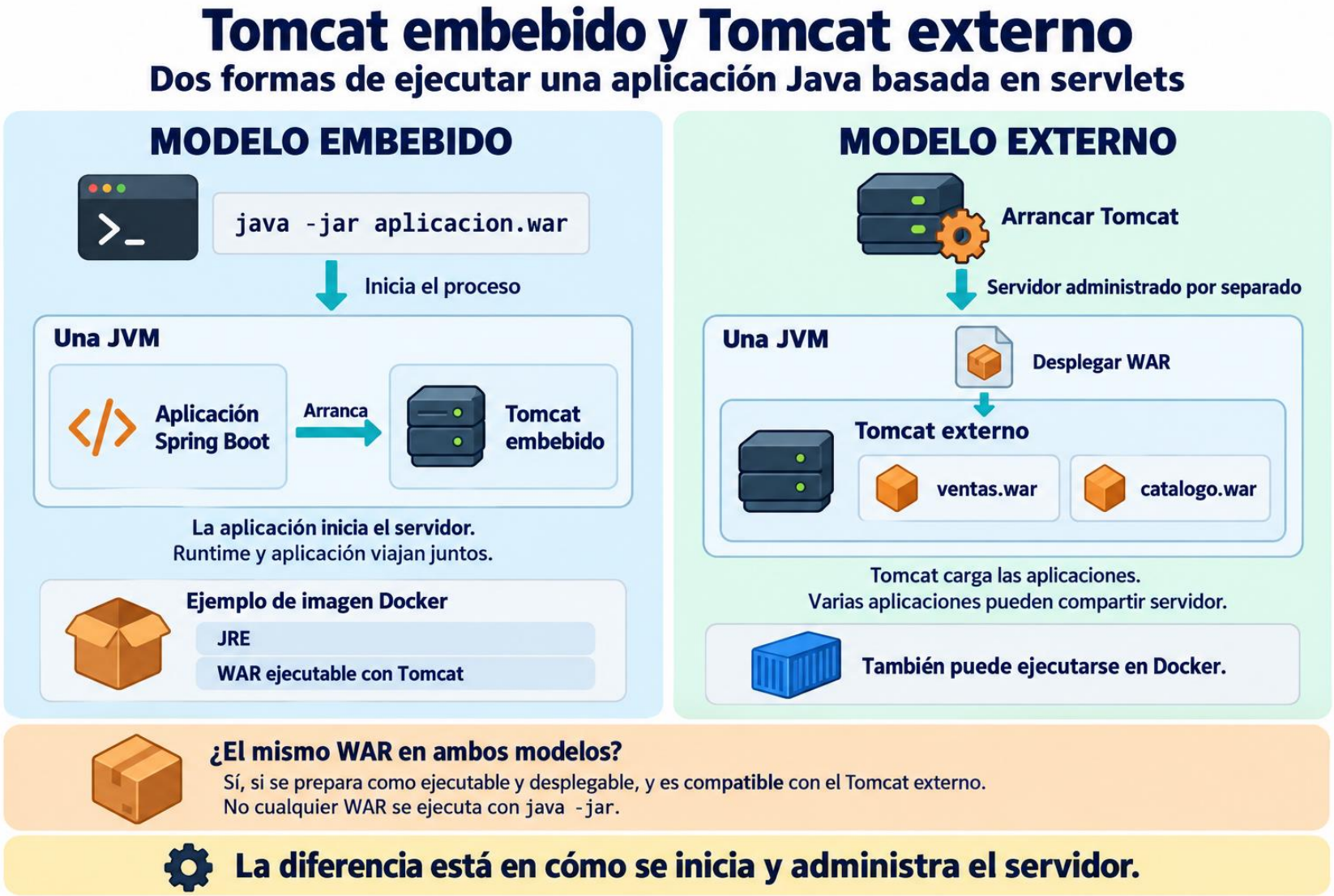
Imagen y ejecución

La imagen incorpora el entorno y los ficheros necesarios.

El contenedor ejecuta procesos a partir de esa imagen.

El artefacto de aplicación y la unidad de ejecución cumplen funciones diferentes.

Tomcat embebido y externo



Un WAR preparado para ambos modelos

```
java -jar aplicacion.war
```

Despliegue externo: inicialización y dependencias adecuadas.

También existen aplicaciones con Tomcat embebido en un JAR ejecutable.

Este comando requiere un WAR ejecutable preparado para ese uso.

La compatibilidad entre aplicación, Java y Tomcat sigue siendo necesaria.

Ficheros y directorios de Tomcat

```
tomcat/
├── conf/
│   ├── server.xml
│   ├── context.xml
│   └── tomcat-users.xml
├── lib/
└── webapps/
```

server.xml: conectores y estructura.

context.xml: valores de los contextos.

tomcat-users.xml: usuarios y roles según el mecanismo utilizado.

lib/: bibliotecas compartidas.

webapps/: aplicaciones.

Estructura orientativa de una instalación sencilla. Las ubicaciones pueden configurarse.

WAR, contexto y URL



Diagnóstico de un 404

Ruta del ejemplo

El proxy envía /api/productos.

El endpoint está en /ventas/api/productos.

La petición no llega a ese endpoint.

Comprobaciones

Ruta que recibe y reenvía el proxy.

Contexto desplegado en Tomcat.

Rutas, enlaces y redirecciones de la aplicación.

Contexto, ruta de la aplicación y ruta enviada por el proxy deben encajar.

Identificador de sesión en la cookie

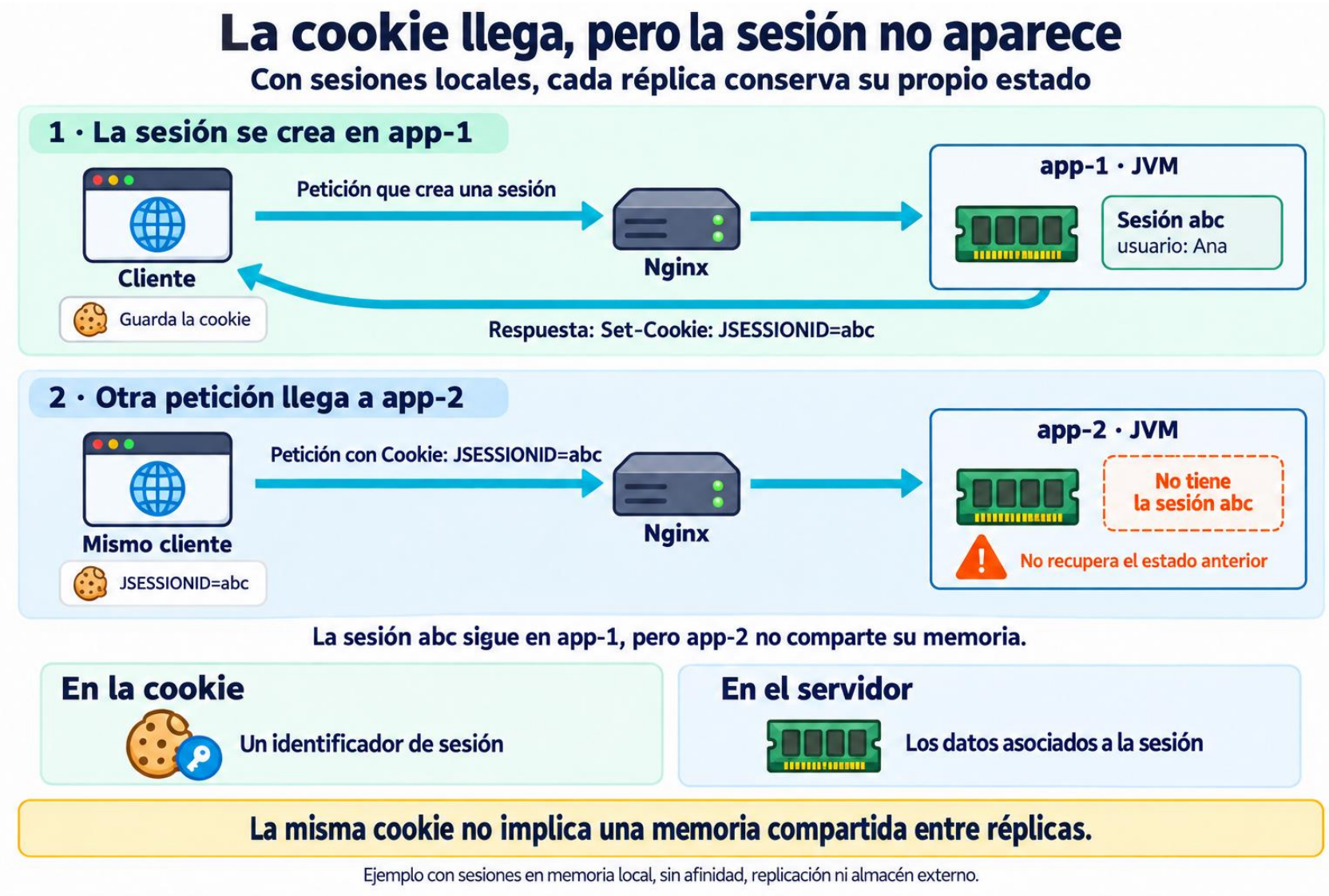
```
Respuesta:
Set-Cookie: JSESSIONID=abc

Petición posterior:
Cookie: JSESSIONID=abc
```

Cabeceras simplificadas. El identificador real es opaco y la cookie tiene atributos de protección y envío.

En este modelo, los datos de la sesión se conservan en el servidor.

Sesión local y cambio de réplica



Estrategias de sesiones



Spring Session y Redis

Integración

La aplicación utiliza HttpSession.

Spring Session integra el repositorio externo.

Las réplicas comparten una configuración compatible.

Condiciones y límites

Añadir Redis exige configurar la aplicación.

Las sesiones pueden expirar o invalidarse.

El nombre de la cookie puede cambiar.

Cambiar de réplica funciona si la sesión sigue siendo válida y el almacén está disponible.

Alcance del estado compartido

Sesiones

La afinidad conserva el estado local.

La replicación necesita coordinación.

Un almacén externo necesita disponibilidad y capacidad.

Otros datos

Los ficheros no se comparten al externalizar sesiones.

Las cachés locales pueden seguir siendo distintas.

Persistir una sesión en disco no la comparte entre réplicas.

Cada tipo de estado necesita una decisión de almacenamiento.

Latencia, throughput y concurrencia

Latencia, throughput y concurrencia

Tres preguntas distintas sobre el mismo sistema

LATENCIA



¿Cuánto tarda una petición?

180 ms

Latencia media del ejemplo

Tiempo desde el inicio hasta la finalización.

THROUGHPUT



¿Cuántas peticiones terminan?

150 peticiones/s

Tasa media de finalización

Trabajo completado por unidad de tiempo.

CONCURRENCIA



¿Cuántas peticiones hay en curso?

27 peticiones

Concurrencia media del ejemplo

Incluye peticiones en ejecución y en espera.

Cómo se relacionan en condiciones estables

Concurrencia media ≈ throughput medio × latencia media

150 peticiones/s × 0,18 s = 27 peticiones

Mismo sistema, mismo periodo y magnitudes medias.

180 ms = 0,18 s

Concurrencia ≠ usuarios virtuales

Un usuario puede esperar, pensar o lanzar varias peticiones.

Concurrencia ≠ capacidad

Más trabajo en curso no añade CPU y puede aumentar las colas.

Tiempo de respuesta, trabajo completado y trabajo en curso no son lo mismo.

Relación de Little: ejemplo con flujo estable y límites de medida coherentes.

Relación entre las magnitudes

```
150 peticiones/s × 0,18 s = 27 peticiones
```

No implica que haya exactamente 27 peticiones en cada instante.

No se sustituye la latencia media por el p95.

La concurrencia incluye ejecución y espera.

180 ms = 0,18 s. El resultado es una concurrencia media.

La relación utiliza medias, un flujo estable y el mismo ámbito de medida.

Throughput, p95 y errores

Qué observar

Peticiones/s: trabajo completado.

p95: umbral de la distribución de latencia.

Errores: recuento y porcentaje según el criterio de la prueba.

Cómo comparar

Mismas rutas y selección de respuestas.

Mismo ámbito y unidades.

Muestras suficientes.

No promediar los p95 de las réplicas.

Una respuesta rápida con errores no demuestra una mejora útil.

Réplicas y recursos compartidos

Tres réplicas no son tres máquinas

Más procesos pueden seguir compitiendo por los mismos recursos



3 procesos ≠ 3 veces más recursos



La caída del host afecta a las tres réplicas.

Comparar 1 y 3 réplicas con criterio

Mantener comparable
- Mismo host, aplicación y datos
- Misma ruta y escenario de carga
- Calentar y repetir las pruebas

Medir conjuntamente
- Peticiones completadas/s
- Latencia p95
- Errores

Interpretar el resultado
- Buscar el recurso limitante
- Considerar la red y el generador
- Concluir solo para esas condiciones

⚠ **Mismos usuarios virtuales no garantizan las mismas peticiones por segundo.**

Replicar puede ayudar, pero la mejora depende del cuello de botella.

El cuello de botella determina la mejora

Qué pueden facilitar

Repartir trabajo entre procesos.

Aprovechar capacidad disponible.

Sustituir copias progresivamente.

Qué puede limitar

CPU o memoria del host.

Disco y red compartidos.

Consultas o conexiones a la base de datos.

Dependencias externas.

Más réplicas también pueden consumir más memoria y abrir más conexiones.

Preparación de una comparación

Mismo escenario

Host y recursos asignados.

Aplicación, datos y dependencias.

Ruta, método y contenido.

Generador, duración y comprobaciones.

Mismo procedimiento

Aplicar un calentamiento equivalente.

Separarlo del periodo medido.

Repetir las ejecuciones.

Conservar todos los resultados.

Cambiamos el número de réplicas y mantenemos comparables las demás condiciones.

Modelos de carga

Modelo cerrado

Cada usuario espera a terminar su iteración.

Una respuesta más lenta puede reducir la tasa de nuevas peticiones.

Modelo abierto

Programa una tasa de inicio de trabajo.

El generador necesita capacidad para sostenerla.

Hay que revisar el trabajo que no consigue iniciar.

Mismos usuarios virtuales no garantizan las mismas peticiones por segundo.

Ámbito y límites de la medición

Métrica elegida

En k6, http_req_duration suma envío, espera y recepción.

No incluye todos los tiempos previos de conexión.

Hay que conocer el criterio de error.

Generador y red

Otro equipo evita competir directamente por la CPU medida.

Su capacidad y la red también pueden limitar el resultado.

Cliente, proxy y aplicación pueden medir intervalos diferentes.

Interpretación de los resultados

Resultado que se documenta

Configuración y escenario.

Peticiones/s, p95 y errores.

Variación entre repeticiones.

Observaciones sobre recursos y red.

Alcance de la conclusión

Explicar cómo se comportaron una y tres réplicas.

Relacionar el cambio con un posible límite.

Evitar convertirlo en una cifra universal de usuarios.

La prueba permite concluir únicamente dentro de las condiciones observadas.

Cierre del tema 3

Qué debemos poder explicar y comprobar

Qué administra Tomcat y cómo se despliega la aplicación.

Cómo encajan el WAR, el contexto y la ruta del proxy.

Dónde vive la sesión y cómo acceden a ella las réplicas.

Qué describen latencia, throughput y concurrencia.

Qué permite concluir una comparación de carga.

Publicar una aplicación exige comprender su ejecución, su estado y sus límites.